JSPM's
RAJARSHI SHAHU COLLEGE OF ENGINEERING
TATHAWADE, PUNE-33
(An Empowered Autonomous Institute Affiliated to Savitribai Phule Pune University, Pune)

Mini Project

on

"Machine Learning"

Phishing Email Detector using ML

Name	PRN	Roll Number
Prakhyat Raj Gupta	RBT24CS100	CS3230

Table of Contents

1. Abstract 3
2. Introduction 3
3. Literature Review 3
4. Problem Statement 4
5. Objectives 4
6. System Architecture 5
7. Dataset Description 6
8. Methodology 6
8.1 Data Collection and Integration 6
8.2 Text Preprocessing Pipeline 7
8.3 Feature Extraction (TF-IDF) 7
8.4 Machine Learning Models 8
8.5 Ensemble Strategy 8
8.6 Evaluation Metrics 9
9. Implementation Details 9
9.1 Project File Structure 9
9.2 Database Schema 9
9.3 Training Pipeline 10
9.4 Streamlit Web Application 10
10. Results and Analysis 11
10.1 Model Comparison 11
10.2 Per-Source Evaluation 12
10.3 Leave-One-Source-Out Evaluation 12
10.4 Top Discriminating Features 13
11. Screenshots / GUI Description 14
12. Limitations and Future Work 15
12.1 Limitations 15
12.2 Future Work 16
13. Conclusion 16
14. References 17

1. Abstract

Phishing emails remain one of the most prevalent and damaging cyber threats, targeting individuals and organisations through deceptive messages designed to steal credentials, financial information, and sensitive data. This project presents a comprehensive phishing email detection system built using Python 3, combining Natural Language Processing (NLP) techniques with supervised machine learning.

The system processes a corpus of 82,486 emails from six publicly available datasets (CEAS 2008, Enron, Ling, Nazario, Nigerian Fraud, and SpamAssassin), applies a rigorous text-preprocessing pipeline, and trains five classification models: Logistic Regression, Support Vector Machine (Linear SVC), Random Forest, AdaBoost, and a Soft-Voting Ensemble.

The best individual model (SVM) achieves **98.87% accuracy** and **99.92% ROC-AUC**, while the ensemble achieves the lowest false-negative rate of **0.88%**. A cross-source evaluation reveals dataset-specific biases that would affect real-world generalisation. The project includes an interactive Streamlit web application that lets users analyse arbitrary emails in real time, view model comparison dashboards, explore the dataset, and understand the model's decisions through feature-level explanations and rule-based red-flag detection.

2. Introduction

Phishing is a form of social engineering attack in which adversaries impersonate trusted entities through electronic communication, primarily email, to deceive recipients into revealing personal information, clicking malicious links, or downloading malware. According to the Anti-Phishing Working Group (APWG), phishing attacks have consistently broken records year over year, with the FBI's Internet Crime Complaint Center (IC3) reporting billions of dollars in losses annually.

Traditional email security relies on blacklists, sender reputation, and handcrafted rules. While effective against known threats, these approaches struggle with novel phishing campaigns that use previously unseen domains, obfuscated URLs, and sophisticated social engineering language. Machine learning offers a complementary approach by learning discriminative patterns from the email content itself, enabling detection of previously unseen attacks based on linguistic and structural features.

This project implements an end-to-end phishing email detection pipeline that encompasses data ingestion, text preprocessing, feature engineering via TF-IDF vectorization, training and evaluation of multiple supervised classifiers, ensemble methods, cross-source generalisation testing, and an interactive web-based application for real-time analysis and model interpretability.

3. Literature Review

Email phishing detection has been extensively studied in the cybersecurity and machine learning literature. Key approaches include:

Content-Based Analysis. Fette et al. (2007) and Bergholz et al. (2010) demonstrated that statistical analysis of email body text, subject lines, and embedded URLs can effectively distinguish phishing from legitimate emails. Bag-of-words and TF-IDF representations remain popular due to their simplicity and effectiveness on text classification tasks.

Machine Learning Classifiers. Abu-Nimeh et al. (2007) compared Logistic Regression, SVM, Random Forest, and Neural Networks on phishing email datasets, finding that ensemble methods consistently outperformed individual classifiers. Islam and Abawajy (2013) showed that SVM with linear kernels offers a strong balance of accuracy and computational efficiency for text classification.

Feature Engineering. Toolan and Carthy (2010) explored structural email features such as HTML tag counts, embedded image counts, and URL characteristics. However, purely content-based features (word frequencies, n-grams) have shown competitive performance without requiring header analysis.

Deep Learning Approaches. Recent work by Alhogail and Alsabih (2021) applied LSTM and transformer-based models (BERT) to phishing detection, achieving marginal improvements over traditional ML methods at significantly higher computational cost. For educational and resource-constrained deployments, classical ML remains practical.

Dataset Bias. Radev (2008) and subsequent analyses highlighted that phishing detection datasets often conflate spam with phishing and contain source-specific vocabulary that inflates accuracy estimates. Cross-source evaluation is essential to assess true generalisation.

4. Problem Statement

Given a raw email consisting of a subject line and body text, classify it as either phishing (label = 1) or legitimate (label = 0) using supervised machine learning, while providing interpretable explanations of the classification decision and evaluating the model's ability to generalise across heterogeneous email sources.

5. Objectives

- Collect, clean, and integrate six publicly available email datasets into a unified SQLite database.
- Design a robust, reproducible text-preprocessing pipeline that normalises raw email text for ML consumption.
- Extract discriminative features using TF-IDF vectorization with unigram and bigram n-grams.
- Train and evaluate five supervised classifiers: Logistic Regression, Linear SVM, Random Forest, AdaBoost, and a Soft-Voting Ensemble.
- Perform per-source evaluation and leave-one-source-out cross-validation to assess generalisation.

- Build an interactive Streamlit web application with real-time email analysis, model comparison dashboards, dataset exploration, and prediction history.
- Provide model interpretability through feature contribution analysis and rule-based heuristic red-flag detection.

6. System Architecture

The system follows a modular pipeline architecture with clear separation of concerns between data ingestion, preprocessing, model training, and user interface. The pipeline consists of four main stages.

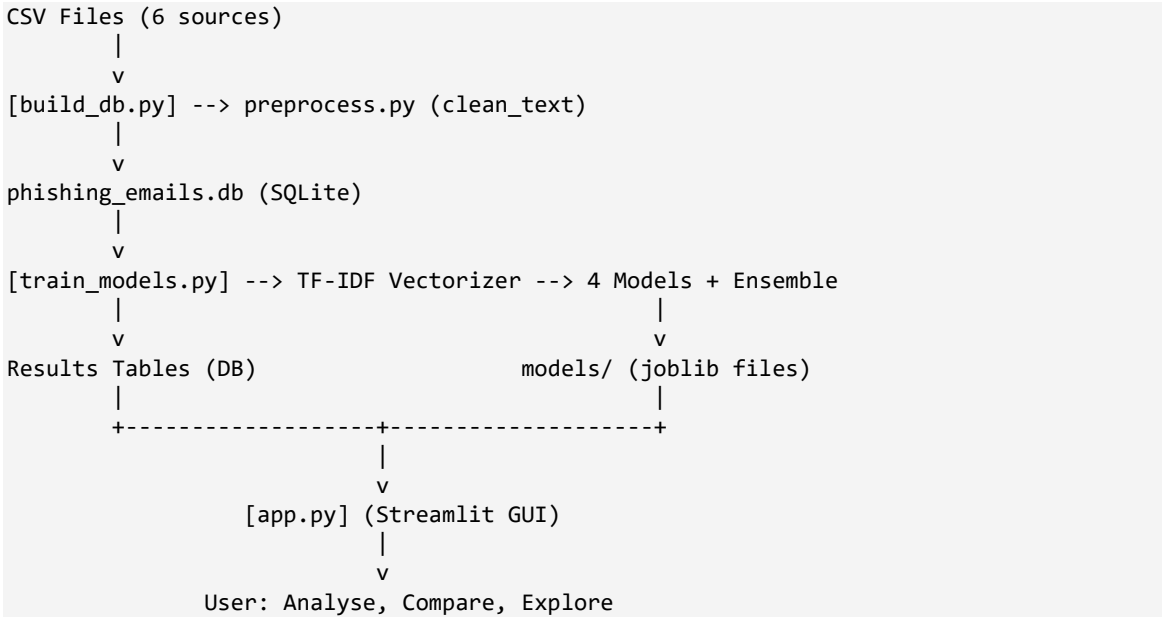
Stage 1: Data Ingestion (`build_db.py`). Reads six CSV source files, handles encoding issues, merges datasets with source tracking, applies the cleaning pipeline, removes short and duplicate texts, and writes the cleaned corpus to an SQLite database.

Stage 2: Model Training (`train_models.py`). Loads cleaned data from SQLite, performs stratified train/test splitting, fits a TF-IDF vectorizer on training data only, trains four individual models and an ensemble, evaluates all models on the test set, performs per-source and cross-source evaluation, and persists models and results.

Stage 3: Web Application (`app.py`). A Streamlit-based GUI with five pages: email detection, model comparison, preprocessing visualisation, dataset exploration, and prediction history.

Stage 4: Shared Components. `common.py` provides paths, DB connections, and utility functions. `preprocess.py` is the single source of truth for text cleaning, used by both training and inference. `ensemble.py` defines the picklable `SoftVotingEnsemble` class.

System Flow



7. Dataset Description

The project uses six publicly available email datasets from the Kaggle "Phishing Email Dataset" collection. These datasets span different time periods, email types, and phishing strategies, providing diversity in the training corpus.

Source	Total Rows	Legitimate (0)	Phishing (1)	Phishing %
CEAS_08	32,195	17,067	15,128	47.0%
Enron	28,319	14,406	13,913	49.1%
SpamAssasin	5,669	4,075	1,594	28.1%
Nigerian_Fraud	3,231	0	3,231	100.0%
Ling	2,854	2,397	457	16.0%
Nazario	1,519	0	1,519	100.0%
Total	**73,787**	**37,945**	**35,842**	**48.6%**

CEAS_08. CEAS 2008 spam challenge dataset with sender, receiver, date, subject, body, label, and URL count. Contains a mix of legitimate and spam/phishing emails.

Enron. Emails from the Enron corporation corpus, widely used in NLP research. Contains corporate communications (legitimate) and spam. Dates primarily from 2000-2001.

Ling. Linguistics academic mailing list emails. Predominantly legitimate academic discussions with a small proportion of spam.

Nazario. Phishing emails collected by security researcher Jose Nazario. All emails are phishing (label = 1), including credential harvesting attempts impersonating banks and online services.

Nigerian_Fraud. Nigerian 419 advance-fee fraud emails dating from 1998-2007. All emails are phishing/scam (label = 1), characterised by appeals to greed and urgency.

SpamAssasin. The SpamAssassin public email corpus, containing both ham and spam emails used for training spam filters.

Data cleaning removed 518 rows with fewer than 3 tokens after preprocessing and 8,181 duplicate texts, reducing the corpus from 82,486 to 73,787 unique email records.

8. Methodology

8.1 Data Collection and Integration

The six CSV files use two different schemas: a 3-column format (subject, body, label) for Enron, Ling, and SpamAssasin; and a 7-column format (sender, receiver, date, subject, body, label/urls) for CEAS_08, Nazario, and Nigerian_Fraud. The column ordering of label and urls also varies between files.

The `load_sources()` function in `preprocess.py` normalises all column names to lowercase, extracts only the common columns (subject, body, label), fills missing values with empty strings, adds a `source` column for provenance tracking, and concatenates all DataFrames. Sender, receiver, date, and URL columns are deliberately excluded as they leak the data source and are missing in several files.

8.2 Text Preprocessing Pipeline

The `clean_text()` function in `preprocess.py` serves as the single source of truth for text normalisation, used identically during training and inference. The pipeline applies the following steps in order:

1. **Truncation.** Truncate raw text to the first 5,000 characters to limit memory usage and processing time for extremely long emails.
2. **HTML cleaning.** Apply `html.unescape()` to decode HTML entities (e.g., `&` to &), then strip all HTML tags using the regex `<[^>]+>`.
3. **URL and email normalisation.** Replace URLs (http/https/www patterns) with the token `urltoken` and email addresses with `emailtoken`. This preserves the structural signal (presence of links/addresses) without memorising specific domains.
4. **Lowercasing.** Convert all text to lowercase for case-insensitive matching.
5. **Digit normalisation.** Replace sequences of digits with `numtoken`, capturing the presence of numbers (account IDs, amounts) without overfitting to specific values.
6. **Character filtering.** Remove all non-alphabetical characters and collapse multiple whitespace characters into single spaces.
7. **Token length filtering.** Remove tokens shorter than 2 characters or longer than 25 characters to eliminate noise and artefacts.
8. **Stop word removal.** Handled by `TfidfVectorizer` with `stop_words='english'`, removing common English function words (the, is, at, etc.) that carry no discriminative signal.

After cleaning, rows with fewer than 3 remaining tokens are dropped (518 rows), and duplicate clean texts are removed keeping the first occurrence (8,181 duplicates).

8.3 Feature Extraction (TF-IDF)

Term Frequency-Inverse Document Frequency (TF-IDF) is used to convert cleaned text into numerical feature vectors. TF-IDF weights terms by how frequently they appear in a document (TF) and how rare they are across the corpus (IDF), giving higher weights to terms that are distinctive to a document.

Parameter	Value	Purpose
`max_features`	30,000	Maximum vocabulary size
`ngram_range`	(1, 2)	Unigrams and bigrams

Parameter	Value	Purpose
`min_df`	3	Minimum document frequency (ignore very rare terms)
`max_df`	0.9	Maximum document frequency (ignore very common terms)
`sublinear_tf`	True	Apply 1 + log(tf) scaling
`stop_words`	english	Remove standard English stop words

The vectorizer is fitted exclusively on the training set to prevent data leakage. The test set is transformed using the fitted vocabulary. The resulting feature matrix has shape (59,029 x 30,000) for training and (14,758 x 30,000) for testing.

8.4 Machine Learning Models

Four individual supervised classifiers are trained.

1. Logistic Regression. A linear classifier that models the log-odds of the phishing class as a linear combination of TF-IDF features. Uses the liblinear solver optimised for sparse data, with L2 regularisation (C = 1). Fast to train and highly interpretable through feature coefficients.

2. Support Vector Machine (Linear SVC). A maximum-margin linear classifier using `LinearSVC` with C = 1 regularisation. Wrapped in `CalibratedClassifierCV(cv=3)` to produce probability estimates via Platt scaling. A linear kernel is used instead of RBF because the RBF kernel has $O(n^2)$ training complexity, making it computationally infeasible for 60,000+ sparse TF-IDF vectors with 30,000 features.

3. Random Forest. An ensemble of 200 decision trees, each trained on a bootstrap sample with sqrt(n_features) random feature subsets. Captures non-linear feature interactions and is robust to outliers. Uses parallel training (`n_jobs=-1`).

4. AdaBoost. A boosting ensemble of 200 shallow decision trees (`max_depth=2`) with learning rate 0.5. Each subsequent tree focuses on the samples misclassified by previous trees, iteratively improving on hard examples.

8.5 Ensemble Strategy

The Soft-Voting Ensemble (`SoftVotingEnsemble` class in `ensemble.py`) averages the `predict_proba` outputs of all four individual models with equal weights. The final prediction uses a threshold of 0.5: if the average phishing probability exceeds 0.5, the email is classified as phishing.

This approach combines the strengths of different model families (the linear decision boundaries of LR and SVM, the non-linear interactions captured by Random Forest, and the iterative error correction of AdaBoost) without requiring additional hyperparameter tuning on the test set.

8.6 Evaluation Metrics

Metric	Definition
Accuracy	Proportion of correctly classified emails: (TP + TN) / Total
Precision	Of emails flagged as phishing, how many truly are: TP / (TP + FP)
Recall	Of actual phishing emails, how many were caught: TP / (TP + FN)
F1 Score	Harmonic mean of precision and recall: 2 × P × R / (P + R)
ROC-AUC	Area under the Receiver Operating Characteristic curve; measures discrimination ability across all thresholds
FNR	False Negative Rate = FN / (FN + TP); the proportion of phishing emails missed (most critical for security)

9. Implementation Details

9.1 Project File Structure

File / Directory	Description
`requirements.txt`	Python package dependencies (pandas, numpy, scikit-learn, streamlit, matplotlib, joblib)
`common.py`	Shared constants (paths), DB connection helpers, directory management
`preprocess.py`	Text preprocessing pipeline: `load_sources()`, `clean_text()`, `build_clean_dataset()`
`ensemble.py`	`SoftVotingEnsemble` class: picklable soft-voting ensemble
`build_db.py`	Data ingestion script: reads CSVs, cleans, writes SQLite database
`train_models.py`	Training pipeline: TF-IDF, 4 models + ensemble, evaluation, results to DB
`app.py`	Streamlit web application with 5 pages
`README.md`	Project documentation and run instructions
`data/`	Directory containing 6 source CSV files + sanity check file
`models/`	Saved TF-IDF vectorizer and trained model files (.joblib)
`phishing_emails.db`	SQLite database with emails, results, and predictions

9.2 Database Schema

The SQLite database (`phishing_emails.db`) contains seven tables:

Table	Columns	Description
`emails`	id, source, subject, body_trunc, clean_text, label, split	Cleaned email corpus with train/test split assignment
`model_results`	model, accuracy, precision, recall, f1, roc_auc, fnr, train_seconds, confusion_matrix_json	Per-model evaluation metrics

Table	Columns	Description
source_results	model, source, n, accuracy, recall	Per-model, per-source accuracy and recall
cross_source_results	held_out_source, n, accuracy, recall, f1	Leave-one-source-out evaluation
roc_points	model, fpr_json, tpr_json	Downsampled ROC curve data for plotting
top_words	word, coefficient	Top 30 phishing + 30 legitimate features from LR
predictions	id, created_at, subject, text_preview, model_probs_json, ensemble_prob, verdict	Logged user predictions from the GUI

9.3 Training Pipeline

The train_models.py script supports two execution modes:

- python train_models.py --fast: uses a stratified 25,000-row sample for quick iteration.
- python train_models.py: full training on all 73,787 rows.

The training process follows these steps:

1. Load cleaned emails from the SQLite database.
2. Perform a stratified 80/20 train/test split (random_state=42) preserving class balance.
3. Fit the TF-IDF vectorizer on the training set only (no data leakage).
4. Train each model and record training time.
5. Evaluate all models on the test set: accuracy, precision, recall, F1, AUC, FNR, confusion matrix.
6. Evaluate per-source performance for each model.
7. Run leave-one-source-out cross-validation (LR only, for computational efficiency).
8. Extract top discriminating features from LR coefficients.
9. Save models to disk (joblib) and results to SQLite.

9.4 Streamlit Web Application

The Streamlit app (app.py) provides five interactive pages.

Page 1: Detect Email. Users enter a subject and body (or upload .txt/.eml files). Two example buttons pre-fill phishing and legitimate samples. On clicking "Analyse", the app runs clean_text(), transforms via TF-IDF, queries all 5 models, and displays a verdict banner with ensemble probability, an individual model probability bar chart, a model agreement indicator, rule-based red flags (URLs, urgency words, money/prize words, CAPS ratio, link mismatches), and top contributing words from LR coefficients.

Page 2: Model Comparison. Displays the metrics table with best values highlighted, a grouped bar chart of metrics, overlay ROC curves for all models, a confusion matrix heatmap with model selector, and a training time comparison chart.

Page 3: Preprocessing. Lists the 8 cleaning steps, shows a raw vs cleaned example email from the database, and displays dataset statistics (total rows, average body/clean text length, class distribution).

Page 4: Dataset Explorer. Stacked bar chart of phishing vs legitimate per source, text length histogram, per-source accuracy table, leave-one-source-out results with a plain-language explanation, and a read-only SQL query box.

Page 5: Prediction History. Shows total and flagged prediction counts, a table of all past analyses with timestamps and probabilities, and a clear button.

10. Results and Analysis

10.1 Model Comparison

All models were trained on 59,029 emails and evaluated on 14,758 emails (stratified 80/20 split). The following table presents the test-set performance:

Model	Accuracy	Precision	Recall	F1	ROC-AUC	FNR	Time (s)
SVM (LinearSVC)	0.9887	0.9879	0.9888	0.9884	0.9992	0.0112	1.3
Ensemble (Soft Voting)	0.9879	0.9839	0.9912	0.9876	0.9993	0.0088	0.0
Logistic Regression	0.9852	0.9805	0.9891	0.9848	0.9988	0.0109	0.8
Random Forest	0.9803	0.9813	0.9782	0.9797	0.9979	0.0218	19.6
AdaBoost	0.9153	0.8750	0.9632	0.9170	0.9749	0.0368	178.8

Key observations

- SVM (LinearSVC) achieves the highest individual accuracy (98.87%) and F1 score (98.84%), demonstrating the effectiveness of maximum-margin linear classifiers on high-dimensional sparse text features.
- The Soft-Voting Ensemble achieves the lowest false-negative rate (0.88%), meaning it misses fewer phishing emails than any individual model. This is the most critical metric for security applications.
- Logistic Regression offers the best speed-accuracy trade-off, training in just 0.8 seconds while achieving 98.52% accuracy.
- AdaBoost shows the lowest precision (87.50%) due to a higher false-positive rate, but maintains high recall (96.32%), suggesting it aggressively flags suspicious emails.

- Random Forest shows slightly lower recall on minority sources (Ling, Nazario), indicating potential underfitting on underrepresented phishing patterns.
- All models achieve ROC-AUC above 0.97, indicating excellent discrimination ability across all classification thresholds.

10.2 Per-Source Evaluation

Per-source evaluation reveals how each model performs on emails from different datasets, highlighting potential source-specific biases.

Source	Test Emails	Accuracy	Recall
CEAS_08	6,477	0.9937	0.9977
Enron	5,654	0.9841	0.9875
SpamAssasin	1,103	0.9755	0.9676
Nigerian_Fraud	683	0.9985	0.9985
Ling	527	0.9734	0.9425
Nazario	314	0.9809	0.9809

The ensemble performs consistently well across all sources, with accuracy above 97% for all datasets. Nigerian_Fraud emails are the easiest to detect (99.85% recall) due to their distinctive 419 scam language. Ling shows slightly lower recall (94.25%) as academic linguistic discussions can occasionally contain terms that overlap with phishing vocabulary.

10.3 Leave-One-Source-Out Evaluation

To test whether models learn generalisable phishing patterns versus source-specific vocabulary, a leave-one-source-out evaluation was performed using Logistic Regression. For each source, the model is trained on all other sources and tested on the held-out one.

Held-Out Source	n	Accuracy	Recall	F1
CEAS_08	32,195	0.9074	0.9123	0.9026
Enron	28,319	0.8499	0.8825	0.8525
SpamAssasin	5,669	0.9187	0.9429	0.8670
Nigerian_Fraud	3,231	N/A	0.9929	N/A
Ling	2,854	0.9212	0.9278	0.7903
Nazario	1,519	N/A	0.4878	N/A

Interpretation. When trained on all other sources and tested on Enron, accuracy drops to 84.99%, suggesting the model partially learned Enron-specific corporate vocabulary rather than universal phishing indicators. The most striking result is Nazario, where recall drops to 48.78%, indicating that the targeted credential-harvesting phishing in Nazario is stylistically distinct from the spam/fraud in

the training sources. Nigerian_Fraud maintains 99.29% recall because its advance-fee scam language (urgency, money, inheritance) overlaps strongly with general phishing patterns.

10.4 Top Discriminating Features

The Logistic Regression model's learned coefficients reveal which words are most indicative of phishing (positive coefficients) and legitimate (negative coefficients) emails. The top 15 features in each direction are:

Top 15 phishing indicators

Word / N-gram	Coefficient	Interpretation
account	+5.4404	Credential harvesting language
http	+5.3735	Embedded links to phishing sites
click	+5.3559	Call-to-action to visit malicious links
life	+5.0132	Spam product marketing (insurance, lifestyle)
money	+4.2394	Financial fraud / advance-fee scams
watches	+4.1850	Counterfeit product spam
love	+3.9197	Romance scam / spam
health	+3.6997	Health product spam
men	+3.5507	Targeted product spam
watch	+3.5487	Counterfeit product spam
meds	+3.2068	Pharmaceutical spam
replica	+3.1924	Counterfeit product spam
remove	+3.1840	Unsubscribe link language
bank	+3.1502	Banking phishing attacks
quality	+3.0920	Product spam marketing

Top 15 legitimate indicators

Word / N-gram	Coefficient	Interpretation
enron	-10.9190	Enron corpus-specific (dataset bias)
wrote	-10.3742	Email reply/forward conventions
thanks	-7.5607	Polite sign-off in professional emails
vince	-5.7717	Enron corpus-specific person name (dataset bias)
opensuse	-5.4758	Technical mailing list (Ling dataset)
louise	-5.1846	Enron corpus-specific person name (dataset bias)
university	-5.0746	Academic / institutional context
rant	-4.6032	Casual discussion tone
python	-4.4474	Technical programming discussion

Word / N-gram	Coefficient	Interpretation
numtoken numtoken	-4.3040	Multiple numbers in sequence (timestamps, IDs)
language	-4.2344	Academic linguistics discussion
perl	-4.1363	Technical programming discussion
attached	-4.0975	Business email attachment references
url urltoken	-4.0793	Formatted URL references in legitimate discussion
questions	-4.0674	Professional inquiry language

> **Note:** Several top legitimate features (enron, vince, louise) reflect dataset-specific bias rather than universal indicators of legitimate email. This underscores the importance of the leave-one-source-out evaluation and the limitations of historical datasets for training production phishing detectors.

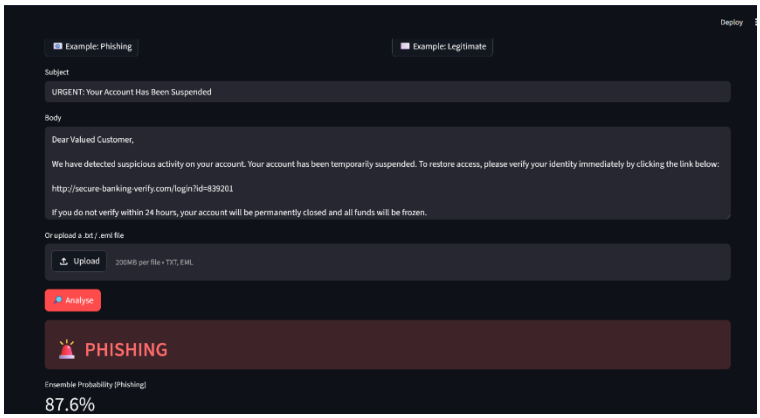
11. Screenshots / GUI Description

The Streamlit web application is launched with the command `streamlit run app.py` and is accessible at http://localhost:8501. The application consists of five pages accessible from a sidebar navigation menu.

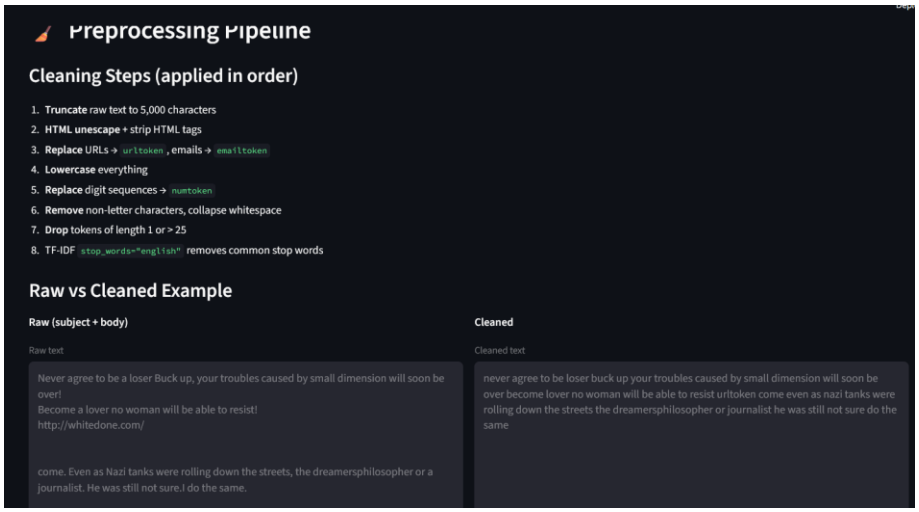
Detect Email page. The main detection page features two pre-filled example buttons (one phishing, one legitimate), text input fields for subject and body, a file uploader for .txt/.eml files, and an "Analyse" button. Upon analysis, the page displays:

- A colour-coded verdict banner (red for phishing, green for legitimate)
- Ensemble probability with a progress bar
- Horizontal bar chart of individual model probabilities (red/green coded)
- Model agreement indicator (e.g., "5/5 models say phishing")
- Rule-based red flags panel with icons
- Top contributing words table with contribution scores

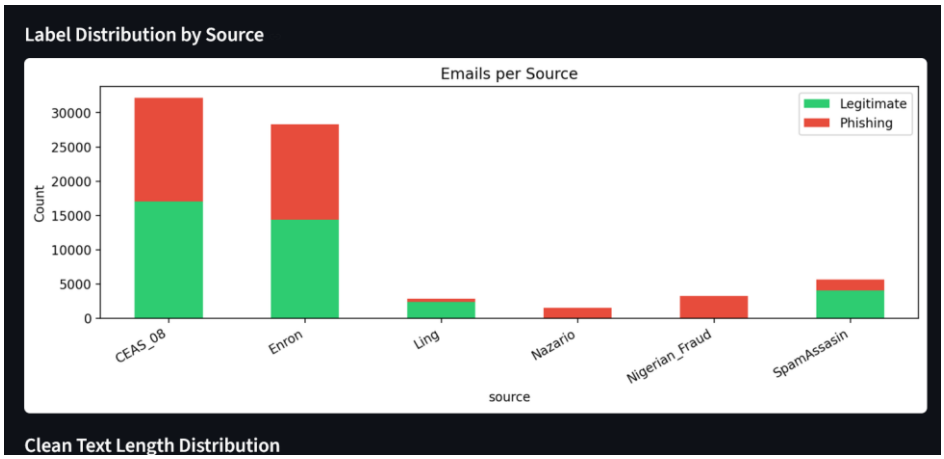
Model Comparison page. Displays all evaluation metrics in a styled table with best values highlighted in green. Below are a grouped bar chart comparing metrics across models, overlay ROC curves for all models on a single plot, a confusion matrix heatmap with a dropdown model selector, and a horizontal bar chart of training times.



Preprocessing page. Documents the 8-step cleaning pipeline, shows a side-by-side raw vs cleaned example email from the database, and presents dataset statistics including total row count, average text lengths, and class distribution.



Dataset Explorer page. Features a stacked bar chart of phishing vs legitimate emails per source, a text length histogram coloured by class, the per-source accuracy/recall table, the leave-one-source-out results table with an explanatory info box, and a read-only SQL query text area for direct database exploration.



Prediction History page. Shows summary metrics (total analyses, number flagged as phishing), a scrollable table of all past predictions with timestamps and probability details, and a clear history button.

12. Limitations and Future Work

12.1 Limitations

Source bias. The models learn source-specific vocabulary (e.g., "enron", "vince") that inflates accuracy on the test set but would not generalise to real-world email streams. The leave-one-source-out evaluation quantifies this effect.

Historical data. The training data spans 1998-2008 (Enron from 2001, Nigerian Fraud from 1998-2007, CEAS from 2008). Modern phishing emails use more sophisticated language, brand impersonation, and URL obfuscation techniques not represented in these historical corpora.

English only. All training data is in English. The system cannot reliably detect phishing in other languages without multilingual training data and potentially different feature representations.

Spam vs phishing conflation. The datasets label all spam as phishing (label = 1). True phishing (credential theft, malware delivery) is a specific subset of spam. This conflation means the model detects spam broadly rather than phishing specifically.

No header analysis. The system uses only subject and body text. Real-world phishing detection benefits from email header analysis (SPF/DKIM/DMARC validation, sender reputation, IP geolocation).

Static model. The trained models are not updated as new phishing techniques emerge. A production system would require continuous retraining or online learning.

12.2 Future Work

- Incorporate modern phishing datasets (2020+) with contemporary social engineering tactics.
- Add URL reputation checking and domain age analysis as additional features.
- Implement deep learning models (BERT, RoBERTa) for contextual text understanding.
- Add email header analysis (SPF, DKIM, DMARC, sender reputation) as complementary features.
- Support multilingual phishing detection with language-agnostic embeddings.
- Implement online/incremental learning for model updates without full retraining.
- Add adversarial robustness testing against evasion attacks (paraphrasing, homoglyph substitution).
- Deploy as a real-time email gateway plugin or browser extension.

13. Conclusion

This project demonstrates a complete end-to-end machine learning pipeline for phishing email detection, from raw data ingestion through model training to an interactive web-based deployment. The key findings are:

- TF-IDF vectorization with bigram features provides a strong, efficient representation for email text classification, achieving over 98% accuracy with linear classifiers.
- The Soft-Voting Ensemble achieves the best overall security posture with only 0.88% of phishing emails missed (false-negative rate), combining the strengths of four diverse model families.

- SVM with a linear kernel offers the best individual model performance (98.87% accuracy, 99.92% AUC) while remaining computationally efficient on sparse high-dimensional data.
- Leave-one-source-out evaluation reveals significant source bias in the training data, with accuracy dropping 5-15% when testing on unseen sources, particularly for Enron corporate emails and Nazario targeted phishing.
- The interactive Streamlit application makes the system accessible to non-technical users, providing real-time analysis with interpretable explanations through feature contributions and rule-based red flags.
- While the system achieves excellent performance on the available datasets, the limitations of historical data, source bias, and spam-phishing conflation must be addressed before production deployment.

GithHub Link

https://github.com/prakhyatg17/Phishing-Mail-Detector

14. References

[1] Abu-Nimeh, S., Nappa, D., Wang, X., & Nair, S. (2007). A comparison of machine learning techniques for phishing detection. *Proceedings of the Anti-Phishing Working Groups 2nd Annual eCrime Researchers Summit*, 60-69.

[2] Bergholz, A., De Beer, J., Glahn, S., Moens, M. F., Paaß, G., & Strobel, S. (2010). New filtering approaches for phishing email. *Journal of Computer Security*, 18(1), 7-35.

[3] Fette, I., Sadeh, N., & Tomasic, A. (2007). Learning to detect phishing emails. *Proceedings of the 16th International Conference on World Wide Web*, 649-656.

[4] Islam, R., & Abawajy, J. (2013). A multi-tier phishing detection and filtering approach. *Journal of Network and Computer Applications*, 36(1), 324-335.

[5] Toolan, F., & Carthy, J. (2010). Feature selection for spam and phishing detection. *2010 eCrime Researchers Summit*, 1-12.

[6] Alhogail, A., & Alsabih, A. (2021). Applying machine learning and natural language processing to detect phishing email. *Computers & Security*, 110, 102414.

[7] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

[8] Kaggle. Phishing Email Dataset. https://www.kaggle.com/datasets/subhajournal/phishing-email-dataset

[9] Anti-Phishing Working Group (APWG). Phishing Activity Trends Reports. https://apwg.org/trendsreports/